

// Jivops

Guía completa — 30 lecciones

Arquitectura · Roles · Idempotencia · Seguridad · Escalabilidad · CI/CD

Contenido

Módulo 1: Arquitectura de proyectos

- Lección 01 — Estructura de proyectos Ansible en producción
- Lección 02 — Inventarios dinámicos
- Lección 03 — Variables y precedencia
- Lección 04 — Ansible Vault en profundidad
- Lección 05 — Collections — organización moderna de contenido

Módulo 2: Roles y reutilización

- Lección 06 — Roles reutilizables — buenas prácticas
- Lección 07 — Ansible Galaxy y gestión de dependencias
- Lección 08 — Templates Jinja2 avanzados
- Lección 09 — Handlers y notificaciones
- Lección 10 — Tags para ejecución selectiva

Módulo 3: Idempotencia y errores

- Lección 11 — Idempotencia real — módulos vs shell/command
- Lección 12 — Manejo de errores: block/rescue/always
- Lección 13 — Estrategias de ejecución (linear, free, etc.)
- Lección 14 — Delegación de tareas (delegate_to)
- Lección 15 — Ansible Facts y gathering personalizado

Módulo 4: Seguridad

- Lección 16 — Vault avanzado: múltiples passwords
- Lección 17 — Gestión de secretos externos (HashiCorp Vault)
- Lección 18 — Control de acceso y permisos (become)
- Lección 19 — Auditoría de ejecuciones (logging)
- Lección 20 — Hardening de servidores con Ansible

Módulo 5: Escalabilidad

- Lección 21 — Ansible en paralelo a gran escala (forks, serial)
- Lección 22 — Rolling updates con Ansible
- Lección 23 — Ansible con contenedores (community.docker)
- Lección 24 — Ansible para Kubernetes (kubernetes.core)
- Lección 25 — Optimización de performance de playbooks

Módulo 6: CI/CD y operación

Lección 26 — Testing de playbooks con Molecule

Lección 27 — CI/CD para playbooks Ansible

Lección 28 — Ansible Tower / AWX — introducción

Lección 29 — Versionado y control de cambios de infraestructura

Lección 30 — Proyecto final: automatización completa de un entorno de producción

Módulo 1: Arquitectura de proyectos

Lección 01 — Estructura de proyectos Ansible en producción

Una estructura estándar (roles/, inventories/, group_vars/) facilita que cualquier persona del equipo entienda y mantenga el proyecto. Separar inventories/ por entorno (dev, staging, prod) evita ejecutar algo en el entorno equivocado.

Lección 02 — Inventarios dinámicos

Un inventario dinámico se actualiza automáticamente reflejando la infraestructura real, muy útil en entornos con autoscaling donde nuevos servidores aparecen sin actualizar un archivo a mano.

```
aws_ec2.yml
```

Lección 03 — Variables y precedencia

Las variables pasadas con -e en la línea de comandos tienen mayor precedencia que las del inventario. Conocer este orden es clave para depurar un valor inesperado en producción.

```
ansible-playbook site.yml -e "var=valor"
```

Lección 04 — Ansible Vault en profundidad

Vault protege datos sensibles cifrados dentro de archivos YAML. Nunca se debe subir un archivo con secretos sin cifrar a Git — quedaría expuesto en el historial aunque luego se borre del archivo actual.

```
ansible-vault encrypt secrets.yml
```

Lección 05 — Collections — organización moderna de contenido

Las Collections son un formato empaquetado de roles, módulos y plugins reutilizables. Instalar una collection existente ahorra tiempo frente a escribir un módulo propio desde cero.

```
ansible-galaxy collection install community.general
```

Módulo 2: Roles y reutilización

Lección 06 — Roles reutilizables — buenas prácticas

Un rol bien diseñado es configurable mediante variables, sin valores hardcodeados, lo que permite adaptarlo a distintos proyectos o clientes sin tocar su código.

Lección 07 — Ansible Galaxy y gestión de dependencias

requirements.yml lista las collections y roles externos del proyecto, garantizando que todo el equipo instale exactamente las mismas dependencias de forma reproducible.

```
ansible-galaxy install -r requirements.yml
```

Lección 08 — Templates Jinja2 avanzados

Los templates Jinja2 permiten generar configuración personalizada por servidor a partir de variables, en vez de duplicar archivos fijos idénticos para todos.

```
{{ nombre | upper }}
```

Lección 09 — Handlers y notificaciones

Un handler solo se ejecuta si una tarea lo notifica y hubo un cambio real, evitando reinicios de servicio innecesarios en cada ejecución del playbook.

Lección 10 — Tags para ejecución selectiva

Los tags permiten ejecutar solo un subconjunto de tareas, útil por ejemplo para aplicar un cambio de configuración puntual sin re-desplegar todo el playbook.

```
ansible-playbook site.yml --tags deploy
```

Módulo 3: Idempotencia y errores

Lección 11 — Idempotencia real — módulos vs shell/command

Los módulos específicos de Ansible (como apt) son idempotentes: no aplican el cambio si ya está en el estado deseado. Un `shell: rm -rf ...`` en cambio siempre se ejecuta, sin verificar si ya se hizo antes.

Lección 12 — Manejo de errores: block/rescue/always

'rescue' ejecuta tareas de recuperación si algo del block falla; 'always' se ejecuta siempre. Esto permite recuperarse o limpiar recursos de forma controlada en vez de fallar en seco.

Lección 13 — Estrategias de ejecución (linear, free, etc.)

Con la estrategia 'free', cada host avanza a su propio ritmo sin esperar a los demás, útil cuando las tareas por host son independientes entre sí.

```
strategy: free
```

Lección 14 — Delegación de tareas (delegate_to)

delegate_to ejecuta una tarea concreta en un host distinto al gestionado, por ejemplo para sacar un servidor de un load balancer central antes de actualizarlo.

Lección 15 — Ansible Facts y gathering personalizado

Los facts son información recopilada automáticamente del sistema remoto. Desactivar gather_facts cuando no se necesitan acelera la ejecución del playbook.

```
gather_facts: false
```

Módulo 4: Seguridad

Lección 16 — Vault avanzado: múltiples passwords

Los vault IDs permiten usar distintas contraseñas para distintos entornos cifrados, limitando el impacto si una contraseña de dev se filtra sin comprometer producción.

```
ansible-vault encrypt --vault-id prod@prompt
```

Lección 17 — Gestión de secretos externos (HashiCorp Vault)

Integrar Ansible con un gestor de secretos externo centraliza auditoría y rotación de credenciales, en vez de tener secretos dispersos por proyecto.

Lección 18 — Control de acceso y permisos (become)

become: true eleva privilegios para una tarea concreta. Aplicar el principio de menor privilegio (usarlo solo donde hace falta) reduce el riesgo operativo.

Lección 19 — Auditoría de ejecuciones (logging)

Configurar log_path en ansible.cfg registra cada ejecución, dando trazabilidad de quién ejecutó qué y cuándo — clave para auditorías en producción.

```
[defaults]  
log_path = /var/log/ansible.log
```

Lección 20 — Hardening de servidores con Ansible

Automatizar el hardening (deshabilitar servicios innecesarios, firewall, políticas de contraseñas) garantiza consistencia entre servidores, sin pasos olvidados por error humano.

Módulo 5: Escalabilidad

Lección 21 — Ansible en paralelo a gran escala (forks, serial)

'forks' controla cuántos hosts se procesan en paralelo; 'serial' limita cuántos se actualizan por lote a la vez, reduciendo el riesgo de romper todo simultáneamente.

```
serial: 10
```

Lección 22 — Rolling updates con Ansible

Combinar 'serial' con `delegate_to` (para sacar/meter del balanceador) permite actualizar servidores de a uno mientras el resto sigue sirviendo tráfico, sin downtime total.

Lección 23 — Ansible con contenedores (community.docker)

Esta collection permite gestionar contenedores, imágenes y redes Docker directamente desde un playbook, útil cuando se orquesta infraestructura junto con contenedores.

```
docker_container:
  name: app
  state: started
```

Lección 24 — Ansible para Kubernetes (kubernetes.core)

kubernetes.core permite aplicar manifiestos y gestionar recursos de un cluster K8s desde Ansible, combinando esa gestión con otras tareas de infraestructura en el mismo playbook.

```
k8s:
  state: present
```

Lección 25 — Optimización de performance de playbooks

Activar SSH pipelining reduce el número de conexiones SSH necesarias por tarea, acelerando significativamente la ejecución del playbook.

```
pipelining = True
```


Módulo 6: CI/CD y operación

Lección 26 — Testing de playbooks con Molecule

Molecule prueba roles en entornos aislados (contenedores/VMs) antes de usarlos en producción, detectando errores sin afectar servidores reales.

```
molecule test
```

Lección 27 — CI/CD para playbooks Ansible

Un pipeline típico corre ansible-lint, testing con Molecule y despliegue controlado tras aprobación — detectando problemas temprano, antes de llegar a producción.

Lección 28 — Ansible Tower / AWX — introducción

AWX añade una interfaz web con control de acceso, programación de tareas y auditoría centralizada, permitiendo restringir quién puede ejecutar qué playbooks contra qué entornos.

Lección 29 — Versionado y control de cambios de infraestructura

Versionar playbooks en Git permite revisar cambios, y sobre todo revertir rápido (git revert) al estado anterior conocido si algo rompe producción.

Lección 30 — Proyecto final: automatización completa de un entorno de producción

Se integra todo lo anterior: roles reutilizables, Vault, testing con Molecule, CI/CD, rolling updates y auditoría — el nivel esperado de automatización lista para producción real.